

Quando **l'altro lato della connessione TCP chiude la socket** (inviando un pacchetto FIN o RST) e tu tenti di scriverci sopra usando `write()` o `send()`, il comportamento dipende da come l'altra parte ha chiuso la connessione e da quante volte provi a scrivere.

1. **Primo scenario:** L'altro lato ha fatto una `close()` normale (FIN)

Se l'host remoto ha chiuso la socket in modo pulito (es. ha chiamato `close()` o il processo è terminato), il TCP remoto invia un pacchetto FIN. La connessione è chiusa in ricezione per lui, ma la tua socket non lo sa finché non prova a interagire. Infatti, il pacchetto FIN viene ricevuto anche se il peer invoca `shutdown(..., SHUT_WR)`: una condizione di chiusura parziale in cui non possiamo più ricevere dati dall'altra parte, ma possiamo ancora inviarne.

La prima `write()` o `send()`

- Cosa succede: La scrittura ha successo (restituisce il numero di byte inviati).
- Perché: Il kernel locale si limita a mettere i dati nel buffer di invio e a spedirli.
- Il riscontro: Quando il pacchetto arriva a destinazione, il TCP remoto vede che la socket è chiusa e risponde con un pacchetto RST (Reset).

La seconda `write()` o `send()` (assumendo di aver ricevuto il pacchetto RST)

- Cosa succede: La scrittura fallisce.
- Il segnale SIGPIPE: Il kernel genera un segnale SIGPIPE per il tuo processo. Se non gestisci questo segnale, il tuo programma termina immediatamente (crash).
- L'errore EPIPE: Se il tuo programma ignora il segnale SIGPIPE (o lo blocca), la funzione `write()` o `send()` restituisce -1 e imposta `errno` a EPIPE (Broken pipe).

2. **Secondo scenario:** L'altro lato ha abortito la connessione (RST diretto)

Se l'altro host si è spento improvvisamente, ha crashato, o ha chiuso la socket usando l'opzione `SO_LINGER` impostata a 0, invia direttamente un RST.

- Cosa succede alla prima `write()`: Se il kernel ha già ricevuto e processato il pacchetto RST prima che tu chiamassi la `write()`, la funzione fallirà immediatamente restituendo -1 con `errno` impostato a `ECONNRESET` (Connection reset by peer).

Come gestire la situazione in modo sicuro

Per evitare che il tuo programma crashi a causa di un SIGPIPE improvviso, si usano comunemente due approcci:

Metodo A: Usare i flag della `send()`

Invece di usare `write()`, usa `send()` passando il flag `MSG_NOSIGNAL`. Questo impedisce la generazione del segnale SIGPIPE, e la funzione si limiterà a restituire -1 impostando `errno` = EPIPE.

```
ssize_t bytes_sent = send(sockfd, buffer, len, MSG_NOSIGNAL);  
if (bytes_sent < 0) {  
    if (errno == EPIPE || errno == ECONNRESET) {  
        // Gestisci la disconnessione (es. chiudi la socket locale, pulisci le risorse)  
    }  
}
```

Metodo B: Ignorare il segnale a livello globale

All'inizio del programma (ad esempio nel main), puoi dire al sistema operativo di ignorare il segnale SIGPIPE per l'intero processo:

```
#include <signal.h>  
  
// ...  
  
signal(SIGPIPE, SIG_IGN);
```

In questo modo, qualsiasi successiva write() o send() fallirà restituendo semplicemente -1 con errno = EPIPE, senza far crashare l'applicazione.